Name: Mary-Rose Tracy

ID#: 1001852753

Date: 10/31/2024

Instruction

What the question is

Classic Problems & Class NP

TSP problem given in class:

HOMEWORK

Solve the TSP for 5 cities using this distance matrix:

	B	C	D	E
A	8	4	5	9
B		1	7	3
C			6	2
D				5

Solve the TSP for 5 cities using this dist matrix

	B	C	D	E
A	8	4	5	9
B		1	7	3
C			6	2
D				5

Completing the matrix 5X5 order

	A	B	C	D	E
A	-	8	4	5	9
B	8	-	1	7	3
C	4	1	-	6	2
D	5	7	6	-	5
E	9	3	2	5	-

using Hungarian method: subtract min. element from other elements of respective rows

-	4	0	1	5
7	-	0	6	2
3	0	-	5	1
0	2	1	-	0
7	1	0	3	-

4th col. Does NOT contain any 0 so we subtract 1 (lowest of his col) from other elements.

-	4	0	0	5
7	-	0	5	2
3	0	-	4	1
0	2	1	-	0
7	1	0	2	-

Draw the min. # of horizontal & vertical line to cover all 0's. There are 4 lines (3-horiz, 1 vert) which is less than 5 (order of the matrix) so we subtract two the lowest element from other elements & the elements (non zero) pointing through the lines. & add 1 to the element where the lines intersect.

-	4	1	0	4
6	-	0	4	1
3	0	-	4	1
0	2	2	-	0
6	6	0	2	-

there is also 4 lines. so apply the process previously applied

VII

$$\begin{bmatrix} - & 5 & 2 & 0 & 5 \\ 5 & - & 0 & 3 & 0 \\ 2 & 6 & - & 3 & 0 \\ 0 & 3 & 3 & - & 0 \\ 5 & 0 & 0 & 2 & - \end{bmatrix}$$

there is 5 lines

	A	B	C	D	E
A	-	5	2	[0]	5
B	5	-	[0]	3	0
C	2	0	-	3	0
D	[0]	3	3	-	0
E	5	[0]	[0]	2	-

A → D → A B → C, C → B E, E → B

TOT Dist Covered = 5+5+1+2+3 = 16

another way

	A	B	C	D	E
A	-	5	2	[0]	5
B	5	-	0	3	[0]
C	2	[0]	-	3	6
D	[0]	3	3	-	0
E	5	6	[0]	2	-

A → D → A, B → E, C → B, E → e

TOT_Dist Career = 5+5+3+1+2 = 16

to construct a Loop

A → D → A, B → E → C → B distance = 16

A $\xrightarrow{5}$ D $\xrightarrow{7}$ B $\xrightarrow{3}$ E $\xrightarrow{2}$ C $\xrightarrow{4}$ A Distance = 21

D $\xrightarrow{5}$ A $\xrightarrow{6}$ B $\xrightarrow{3}$ E $\xrightarrow{2}$ C $\xrightarrow{6}$ D Distance = 24

A $\xrightarrow{5}$ D $\xrightarrow{5}$ E $\xrightarrow{2}$ C $\xrightarrow{1}$ B $\xrightarrow{8}$ A Dist = 21

D $\xrightarrow{5}$ A $\xrightarrow{7}$ E $\xrightarrow{2}$ C $\xrightarrow{1}$ B $\xrightarrow{7}$ D Dist = 24

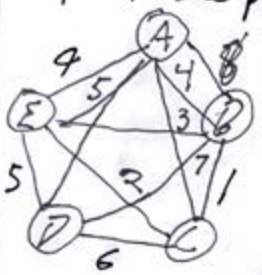
A $\xrightarrow{5}$ D $\xrightarrow{6}$ C $\xrightarrow{1}$ B $\xrightarrow{3}$ E $\xrightarrow{9}$ A Dist = 24

D $\xrightarrow{5}$ A $\xrightarrow{4}$ C $\xrightarrow{1}$ B $\xrightarrow{3}$ E $\xrightarrow{5}$ D Dist = 18

Min Distance Corer = 18 for the root

D → A → C → B → E → D

or I'm pretty sure it can be A→C→B→E→D→A which is also 18. So nonetheless 18,
— D.C (different way to do it)
Choose starting vertex, visit nearest unvisited vertex w/ least edge weight.
Repeat the process. Let's make a matrix



W = weight / Cost

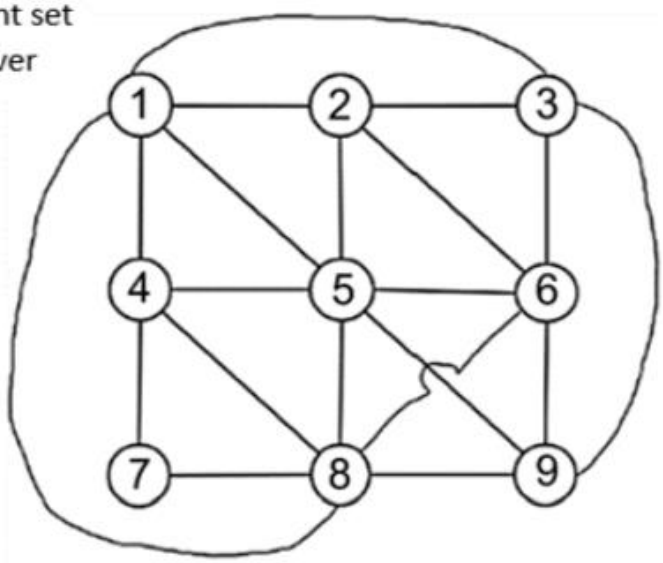
Choose A for starting vertex
nearest C W = 4
B W = 1
E W = 3
D W = 5
A W = 5

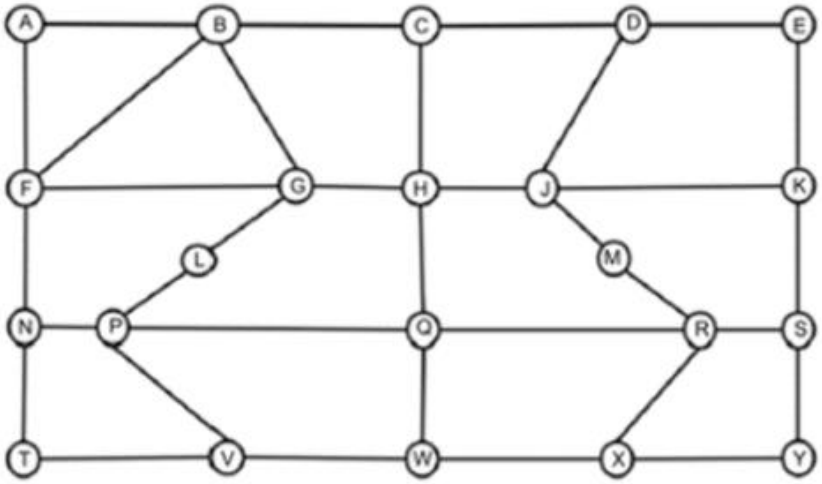
A → C → B → E → D → A
4 + 1 + 3 + 5 + 5 = 18

Classic Problems Worksheet:

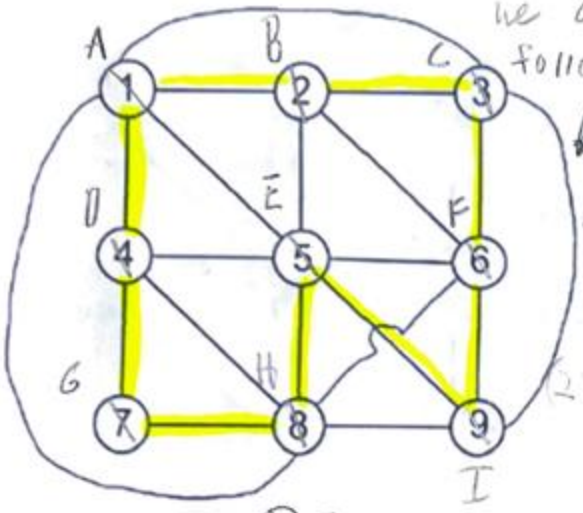
In these graphs, check for:

a) a Hamiltonian cycle
b) the largest clique
c) the largest independent set
d) the smallest vertex cover





A) mm Let's name these letters, b/c I equate #'s to math.

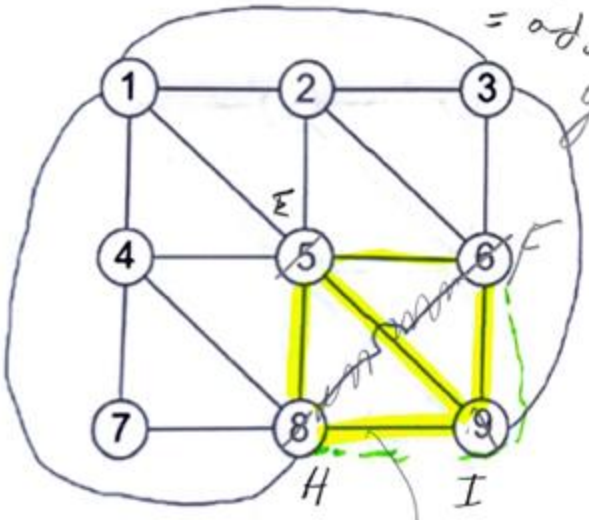


we can cover all verts by following Path

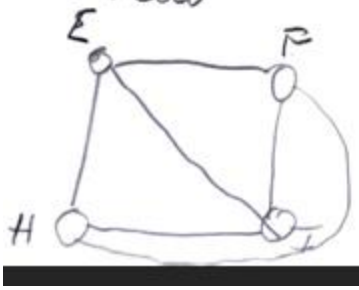
A → B → C → F → I → E → H → G → D → A

Thus, it's hamiltonian cycle.

B) The largest clique possible here = 4. Here, every pair of vert's = adjacent. Also = complete graph

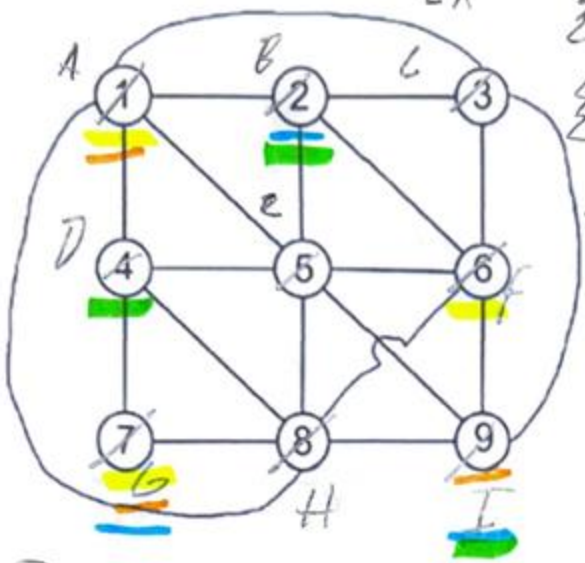


make it look easier to read



3) Largest independent set = of 3 elements

EX



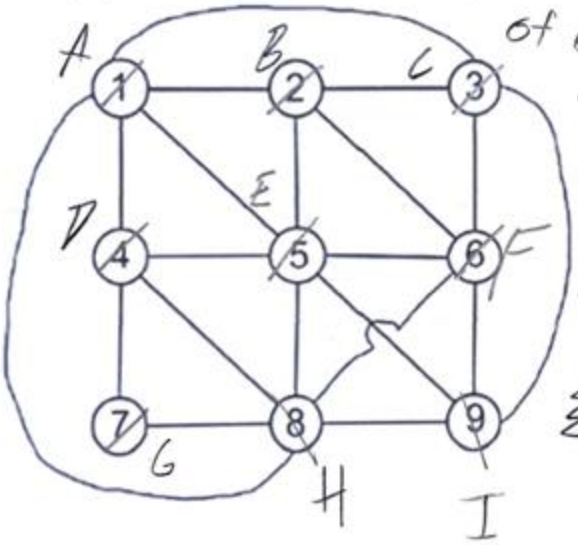
{A, F, G}

{A, I, G}

{B, I, G}

{D, B, I} etc.

4) The smallest vertex cover here is 5. It can be differ sets. but we need min of 6 vertices to cover all the edges.



{E, H, C, B, D, F}

{E, H, C, B, D, I}

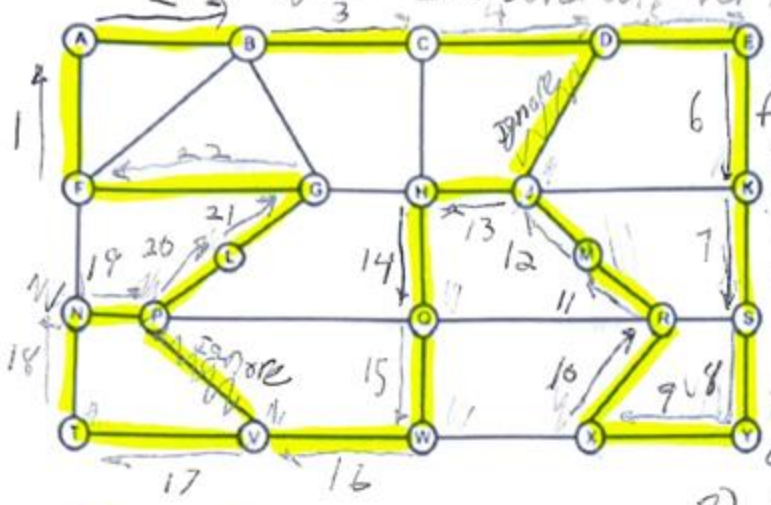
{A, E, D, H, C, F}

7.

A) We can cover all verts by following the Path

f → A → B → C → D → E →
K → S → Y → X → R →
M → J → H → Q → W →
V → T → N → P → L →
G → F. It's

Therefore, Hamiltonian Cycle, steps are 1→22

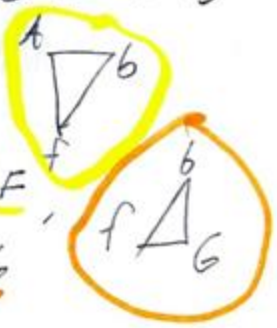


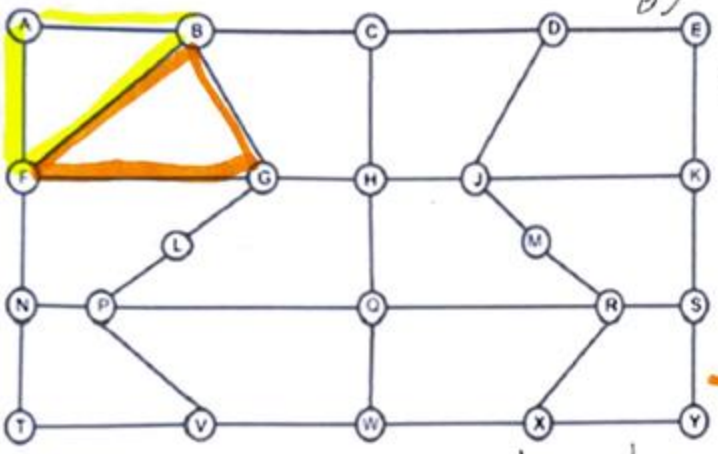
B) The largest Clique here is 3.

So...

A, B, F,

F, B, G





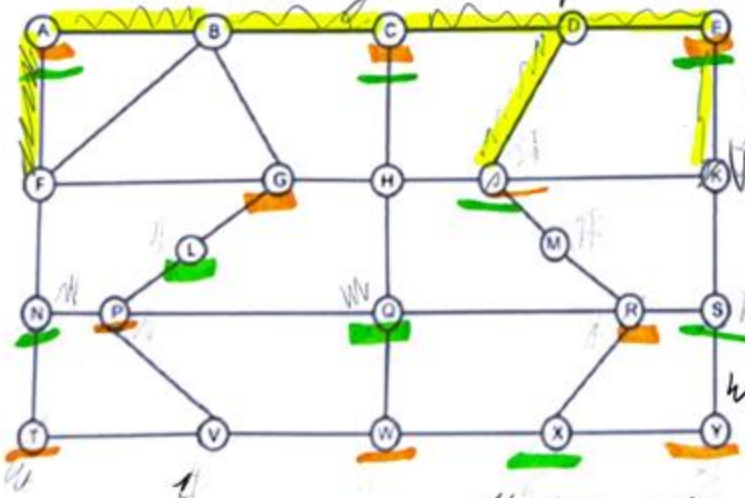
C) Largest Independent set is of 10 vertices.

{A, C, E, G, J, P, R, T, W, Y}

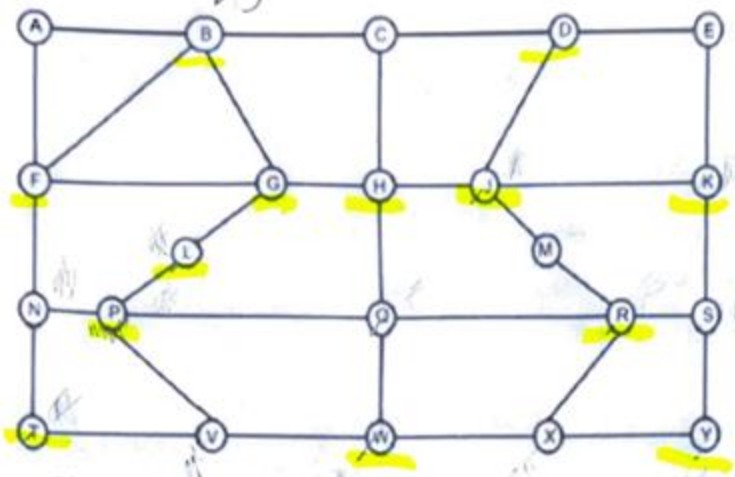
{A, C, E, J, L, N, Q, S, V, X}

No two are adjacent

We can find more combos, but at most there will be 10 vertices.



D)



D) The smallest vertex cover here is 12. There will be different combos. (Like we said previously). Of 12 vert's. However, minimally we need 12 vertices.

So....

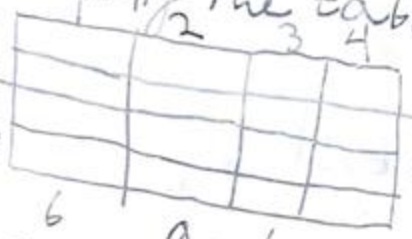
{B, D, F, G, H, J, K, L, R, T, W, Y}

7.4 Fill out the table described in the polynomial time algorithm for context-free language recognition from Theorem 7.16 for string $w =$ `baba` and CFG G:

$$
\begin{aligned}
S &\to RT \\
R &\to TR \mid \texttt{a} \\
T &\to TR \mid \texttt{b}
\end{aligned}
$$

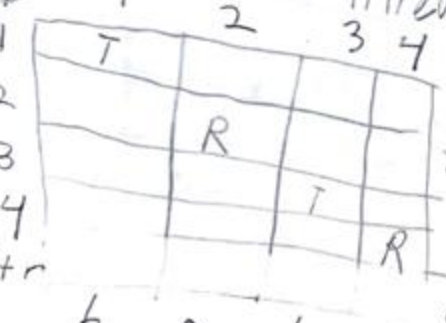
7.4 Take str W = baba. Now we will construct the table for str. baba

Initially the table will be shown like this



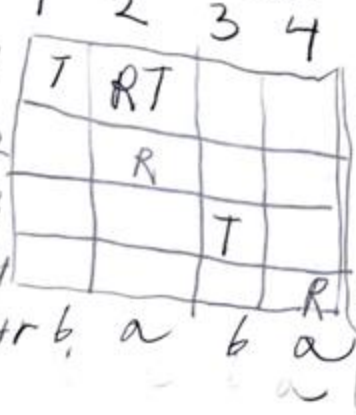
1st all the substr str which are having rules of the $A \to b$ will be occupied in the colat where $i=j$ As per the theorem.

table will be filled like this



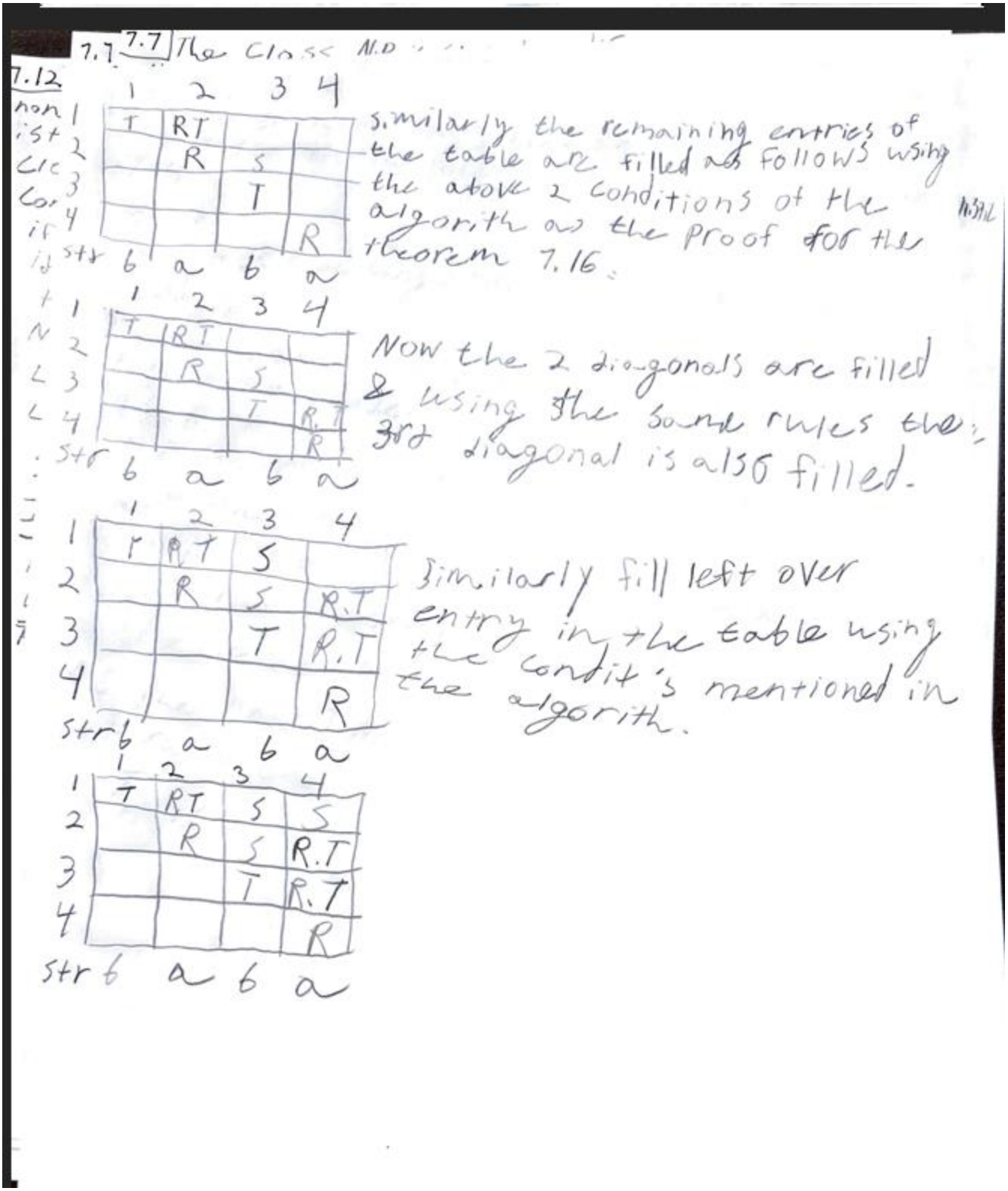
Now fill the remaining entries of the table such that $i \leq j$ & substr formed by 2 or more substr such that rule $A \to BC$. consider the str. ba it's suitable replacement rule.

There is no rule for this str. so the variables of the corresponding individual str. are added to the table



consizer the str. ab for its suitable replacement rule.

similarly fill the next entry in the same row w/ rule $S \to RT$. since it's producing the string ab.



Closure of Class NP: 7.7

7.7 Show that NP is closed under union and concatenation.

7.7 The class NP is closed under union & contat

N.P -class:

NP is a class of lang's that are decidable in nondeterministic polynomial time on a non-deterministic T.M.

Union:

Let A & B be lang's are decided by NP-machines T_A & T_B.

Now we want to show that, there is a non-deterministic Poly time decider T_{AUB} that decides union of A & B.

. The construction of T_{AUB} is as follows:

T_{AUB} == on input S:

1) Run T_A on S. If T_A accepts S, then accept.
2) Else run T_B on S. If T_B accepts S, then accept.
3) Else reject.

As the new T.M T_{AUB} calls T_A & T_B each once, it runs on $O(T_A + T_B)$ as both are NP is T_{AUB}.

7.7 Concat

- Let A & B be lang's are decided by NP-machines T_A & T_B.

Now we want to show that, there is a non-deterministic poly time decider $T_{A \circ B}$ that decides concatenation of A & B.

the construction of $T_{A \circ B}$ is as follows;

$T_{A \circ B}$ = on input S;

1. Split S into S_1, S_2 such that $S = S_1 S_2$
2. Run the NP machine T_A on S_1. If T_A is rejected, then reject.
3. Else run T_B on S_2. If T_B is rejected, then reject.
4. Else accept.

The time taken by step 1 is $O(n)$ in a 2 tape T.M. Thus, T is a poly-time non-deterministic decider for A∘B.

Class NP membership: 7.12

7.12 Call graphs G and H ***isomorphic*** if the nodes of G may be reordered so that it is identical to H. Let $ISO = \{\langle G, H\rangle \mid G \text{ and } H \text{ are isomorphic graphs}\}$. Show that $ISO \in$ NP.

7.12 Class N.P: N.P is a class of Lang's that are non-deterministic polynomial time on a non-deterministic single tape T.M. From the def. 7.19 N.P is the class of Langs that have polynomial time verifiers. Consider... Let ISO = {⟨G, H⟩}. if the nodes of G may be recorded so that it is identical to H then Graphs G & H are said to be isomorphic.

Now it must be proved that ISO ∈ NP

Let $G = (V_G, E_G)$ & $H = (V_H, E_H)$ be the 2 graphs.

Let $V_G = \{u_1, u_2, \ldots u_m\}$, $V_H = \{v_1, v_2, \ldots v_n\}$ be the sets of vertices of G & H.

Isomorphism:

An isomorphism is defined by a mapping $f: V_G \to V_H$ w/ the property that it's a 1-to-1 correspondence. That means it's both 1-to-1 & onto.

This 1-to-1 correspondence is possible only if m=n & for all $u, v \in V_G$ we have $(u,v) \in E_G$ if & only if $(f(u), f(v)) \in E_H$

thus, the correspondence takes edges into edges & non-edges into non-edges.

A mapping f can be represented by the sequence $S = (s_1, s_2, \ldots s_m)$ of indices w/ the property that $f(u_i) = v_{s_i}$, that is i^{th} point of G is mapped into the s_i^{th} point of H.

This sequence S can be taken as certificate.

7.12] Now N is the non-deterministic T.M (NTM) that decides ISO in Polynomial time.

N = on input $\langle\langle G, H\rangle, s\rangle$.

where G & H are graph as defined above s is certificate.

1. Check whether G & H have same # of points.

2. If G & H have same # of points then checks that each pair i, j

$\rightarrow (v_{s_i}, v_{s_j}) \in E_V \ldots (1)$

$\rightarrow (u_i, u_j) \in E_V \ldots (2)$ from 1 & 2

i) E_V can be derived from the above mapping procedure, $f(u_i) = v_{s_i} = E_V$

ii) have $s_i \neq s_j$ & that $(u_i, u_j) \in E_V$ if & only if $(v_{s_i}, v_{s_j}) \in E_V$

iii) if above condit. satisfies, then "accept" 3, otherwise = "reject".

All these checking can be done in time $O(m^2)$, so in time polynomial in the description of (G, H).

Therefore ISO $\in$ NP.

Works Cited:

https://github.com/gaurangsaini/sipser-computation-3rd-solutions/blob/master/Chapter%207/4.pdf

https://github.com/gaurangsaini/sipser-computation-3rd-solutions/blob/master/Chapter%207/7.pdf

https://github.com/gaurangsaini/sipser-computation-3rd-solutions/blob/master/Chapter%207/12.pdf

pdf (questions listed above: 7.4 7.7, 7.12)